

Gabriel Henrique Silva Duque
Rafael Gonçalves Oliveira

**Sistema AutoNível Multimarcas:
Desenvolvimento de Plataforma Web para
Gestão de Concessionária Utilizando
Arquitetura MVC**

BambuÍ – MG
2025

Gabriel Henrique Silva Duque
Rafael Gonçalves Oliveira

Sistema AutoNível Multimarcas: Desenvolvimento de Plataforma Web para Gestão de Concessionária Utilizando Arquitetura MVC

Documentação técnica apresentada como requisito parcial para aprovação na disciplina de Programação Web do Instituto Federal de Minas Gerais – Campus Bambuí.

Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais
Campus Bambuí
Curso de Engenharia de Computação
Disciplina: Programação Web

Bambuí – MG
2025

Sumário

1	INTRODUÇÃO	4
1.1	Objetivos	4
1.1.1	Objetivo Geral	4
1.1.2	Objetivos Específicos	4
1.2	Justificativa	4
2	FUNDAMENTAÇÃO TEÓRICA	5
2.1	Arquitetura MVC	5
2.1.1	Model (Modelo)	5
2.1.2	View (Visão)	5
2.1.3	Controller (Controlador)	5
2.2	Segurança em Aplicações Web	5
2.2.1	Hash de Senhas	5
2.2.2	Prevenção de SQL Injection	5
2.2.3	Controle de Sessões	6
3	TECNOLOGIAS UTILIZADAS	7
4	MODELAGEM DE DADOS	8
4.1	Diagrama Entidade-Relacionamento	8
4.1.1	Tipos Personalizados (ENUMs)	8
4.2	Script de Criação do Banco de Dados	8
5	ARQUITETURA DO SISTEMA	11
5.1	Estrutura de Diretórios	11
5.2	Descrição das Camadas	12
5.2.1	Camada de Configuração (config/)	12
5.2.2	Camada de Controle (Controllers/)	12
5.2.3	Camada de Modelo (Models/)	12
5.2.4	Camada de Visualização (Views/)	12
5.2.5	Diretório Público (public/)	12
6	FUNCIONALIDADES IMPLEMENTADAS	13
6.1	Sistema de Autenticação	13
6.1.1	Cadastro de Usuários	13
6.1.2	Login Seguro	13
6.1.3	Controle de Acesso Baseado em Papéis	13

6.2	Gestão de Veículos	13
6.2.1	Listagem com Filtros	13
6.2.2	Cadastro de Veículos	13
6.2.3	Página de Detalhes	14
6.2.4	Edição e Exclusão	14
6.2.5	Gestão de Categorias	14
6.2.6	Cadastro e Listagem	14
6.2.7	Exclusão de Categorias	14
6.3	Interface do Usuário	14
6.3.1	Design Responsivo	14
6.3.2	Tema Personalizado	14
6.3.3	Interatividade Client-Side	14
7	CONFIGURAÇÃO E INSTALAÇÃO	16
7.1	Requisitos do Sistema	16
7.1.1	Servidor Web	16
7.1.2	Ambiente de Desenvolvimento	16
7.2	Procedimento de Instalação	16
7.2.1	Etapa 1: Clonagem do Repositório	16
7.2.2	Etapa 2: Criação da Estrutura de Diretórios	16
7.2.3	Etapa 3: Configuração do Banco de Dados	17
7.2.4	Etapa 4: Configuração de Credenciais	17
7.2.5	Etapa 5: Inicialização do Servidor	17
8	CONSIDERAÇÕES FINAIS	18
8.1	Contribuições do Projeto	18
8.2	Trabalhos Futuros	18
	REFERÊNCIAS	19

1 Introdução

O presente trabalho descreve o desenvolvimento do sistema **AutoNível Multimarcas**, uma aplicação web destinada ao gerenciamento completo de estoque de veículos, usuários e vendas de uma concessionária automotiva. O projeto foi desenvolvido como trabalho final da disciplina de Programação Web do Instituto Federal de Minas Gerais (IFMG) – Campus Bambuí.

O sistema foi construído utilizando a arquitetura **MVC (Model-View-Controller)**, sem o emprego de *frameworks* robustos, priorizando o domínio de tecnologias nativas e a compreensão aprofundada dos fundamentos da programação web. Esta abordagem permitiu maior controle sobre cada camada da aplicação e proporcionou conhecimento técnico sólido aos desenvolvedores.

1.1 Objetivos

1.1.1 Objetivo Geral

Desenvolver um sistema web funcional e escalável para gerenciamento de concessionária automotiva, aplicando conceitos de arquitetura de software, segurança da informação e boas práticas de desenvolvimento.

1.1.2 Objetivos Específicos

- Implementar sistema de autenticação seguro com hash de senhas;
- Criar operações CRUD completas para gestão de veículos e categorias;
- Desenvolver interface responsiva e intuitiva;
- Aplicar validações tanto no cliente quanto no servidor;
- Utilizar prepared statements para prevenir SQL Injection;
- Implementar controle de acesso baseado em papéis (RBAC).

1.2 Justificativa

A digitalização de processos administrativos é uma necessidade crescente no mercado automotivo. Concessionárias de pequeno e médio porte frequentemente carecem de sistemas acessíveis e customizáveis para gestão de estoque. O AutoNível surge como solução de baixo custo, desenvolvida com tecnologias abertas e de fácil manutenção.

2 Fundamentação Teórica

2.1 Arquitetura MVC

O padrão arquitetural Model-View-Controller (MVC) separa a aplicação em três camadas interdependentes, conforme descrito por gamma1995 e amplamente adotado no desenvolvimento web moderno.

2.1.1 Model (Modelo)

Responsável pela lógica de negócio e acesso aos dados. No contexto do AutoNível, os Models encapsulam todas as operações de banco de dados através de consultas SQL preparadas (Prepared Statements), garantindo segurança contra injeção de código malicioso.

2.1.2 View (Visão)

Camada de apresentação responsável pela renderização da interface do usuário. Utiliza HTML5 semântico combinado com processamento server-side em PHP para exibição dinâmica de dados.

2.1.3 Controller (Controlador)

Atua como intermediário entre Model e View, processando requisições HTTP, validando dados de entrada e coordenando o fluxo de informações entre as camadas.

2.2 Segurança em Aplicações Web

2.2.1 Hash de Senhas

O sistema utiliza a função `password_hash()` do PHP, que implementa o algoritmo bcrypt, considerado seguro pela OWASP (Open Web Application Security Project) para armazenamento de credenciais.

2.2.2 Prevenção de SQL Injection

Todas as consultas ao banco de dados são realizadas através de Prepared Statements com PDO (PHP Data Objects), técnica recomendada pela RFC 7230 para mitigação de ataques de injeção SQL.

2.2.3 Controle de Sessões

Gerenciamento de autenticação utilizando sessões PHP (`$_SESSION`), com validação de tokens e timeout configurável para prevenir sessões expiradas.

3 Tecnologias Utilizadas

O desenvolvimento do sistema AutoNível empregou as seguintes tecnologias:

- **PHP 8.2:** Linguagem de programação server-side com suporte a Orientação a Objetos, type hinting e recursos modernos da linguagem;
- **PostgreSQL 15:** Sistema gerenciador de banco de dados relacional com suporte a tipos enumerados (ENUM) e recursos avançados;
- **HTML5:** Linguagem de marcação semântica para estruturação de conteúdo;
- **CSS3:** Folhas de estilo em cascata com custom properties e flexbox;
- **Bootstrap 5.3:** Framework CSS responsivo para agilidade no desenvolvimento de interfaces;
- **JavaScript ES6+:** Linguagem de programação client-side para interatividade e validações;
- **PDO (PHP Data Objects):** Interface de acesso a banco de dados com prepared statements e suporte nativo ao PostgreSQL.

4 Modelagem de Dados

4.1 Diagrama Entidade-Relacionamento

O banco de dados foi modelado seguindo as formas normais de Codd, garantindo integridade referencial e minimização de redundâncias. As principais entidades são:

- **Usuários:** Armazena dados de autenticação e perfil (inclui CPF e telefone);
- **Clientes:** Registra informações de compradores;
- **Categorias:** Permite a classificação e organização dos tipos de veículos (Ex: SUV, Sedan);
- **Veículos:** Contém especificações técnicas do estoque (inclui campo de quilometragem);
- **Veículos_Fotos:** Relacionamento 1:N com tabela de veículos (suporta foto destaque);
- **Vendas:** Registra transações comerciais.

4.1.1 Tipos Personalizados (ENUMs)

O sistema utiliza tipos enumerados do PostgreSQL para garantir consistência nos dados:

- **tipo_papel_usuario:** Define os papéis ('comum', 'admin');
- **tipo_status_veiculo:** Define os status ('disponivel', 'reservado', 'vendido').

4.2 Script de Criação do Banco de Dados

A seguir, apresenta-se o script SQL completo para criação da estrutura do banco de dados:

```
1 -- PostgreSQL database dump - AutoNivel
2 -- Compativel com: PostgreSQL 12+
3
4 SET statement_timeout = 0;
5 SET lock_timeout = 0;
6 SET client_encoding = 'UTF8';
7 SET standard_conforming_strings = on;
8
9 -- 1. Criacao de Tipos Personalizados (Enums)
10 CREATE TYPE public.tipo_papel_usuario AS ENUM (
11     'comum',
12     'admin'
13 );
```

```
14
15 CREATE TYPE public.tipo_status_veiculo AS ENUM (
16     'disponivel',
17     'reservado',
18     'vendido'
19 );
20
21 -- 2. Tabela: Categorias
22 CREATE TABLE public.categorias (
23     id SERIAL PRIMARY KEY,
24     nome VARCHAR(50) NOT NULL
25 );
26
27 -- 3. Tabela: Clientes
28 CREATE TABLE public.clientes (
29     id SERIAL PRIMARY KEY,
30     nome VARCHAR(100) NOT NULL,
31     cpf_cnpj VARCHAR(20) NOT NULL UNIQUE,
32     telefone VARCHAR(20),
33     email VARCHAR(150),
34     endereco TEXT,
35     data_cadastro TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
36 );
37
38 -- 4. Tabela: Usuarios
39 CREATE TABLE public.usuarios (
40     id SERIAL PRIMARY KEY,
41     nome VARCHAR(100) NOT NULL,
42     email VARCHAR(150) NOT NULL UNIQUE,
43     senha_hash VARCHAR(255) NOT NULL,
44     papel public.tipo_papel_usuario DEFAULT 'comum' NOT NULL,
45     data_criacao TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
46     cpf VARCHAR(20),
47     telefone VARCHAR(20)
48 );
49
50 -- 5. Tabela: Veiculos
51 CREATE TABLE public.veiculos (
52     id SERIAL PRIMARY KEY,
53     marca VARCHAR(50) NOT NULL,
54     modelo VARCHAR(50) NOT NULL,
55     ano_fabricacao INTEGER NOT NULL,
56     ano_modelo INTEGER NOT NULL,
57     cor VARCHAR(30),
58     placa VARCHAR(10) UNIQUE,
59     valor NUMERIC(10,2) NOT NULL,
60     descricao TEXT,
```

```
61     status public.tipo_status_veiculo DEFAULT 'disponivel' NOT NULL,
62     data_cadastro TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
63     km INTEGER DEFAULT 0
64 );
65
66 -- 6. Tabela: Fotos dos Veiculos
67 CREATE TABLE public.veiculos_fotos (
68     id SERIAL PRIMARY KEY,
69     veiculo_id INTEGER NOT NULL,
70     url_foto TEXT NOT NULL,
71     destaque BOOLEAN DEFAULT false
72 );
73
74 -- 7. Tabela: Vendas
75 CREATE TABLE public.vendas (
76     id SERIAL PRIMARY KEY,
77     cliente_id INTEGER NOT NULL,
78     veiculo_id INTEGER NOT NULL UNIQUE,
79     vendedor_id INTEGER NOT NULL,
80     valor_final NUMERIC(10,2) NOT NULL,
81     data_venda TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
82     observacoes TEXT
83 );
84
85 -- 8. Chaves Estrangeiras (Foreign Keys)
86 ALTER TABLE ONLY public.veiculos_fotos
87     ADD CONSTRAINT veiculos_fotos_veiculo_id_fkey
88     FOREIGN KEY (veiculo_id)
89     REFERENCES public.veiculos(id) ON DELETE CASCADE;
90
91 ALTER TABLE ONLY public.vendas
92     ADD CONSTRAINT vendas_cliente_id_fkey
93     FOREIGN KEY (cliente_id)
94     REFERENCES public.clientes(id);
95
96 ALTER TABLE ONLY public.vendas
97     ADD CONSTRAINT vendas_veiculo_id_fkey
98     FOREIGN KEY (veiculo_id)
99     REFERENCES public.veiculos(id);
100
101 ALTER TABLE ONLY public.vendas
102     ADD CONSTRAINT vendas_vendedor_id_fkey
103     FOREIGN KEY (vendedor_id)
104     REFERENCES public.usuarios(id);
```

Listing 4.1 – Script SQL para criação do banco de dados

5 Arquitetura do Sistema

5.1 Estrutura de Diretórios

O projeto segue rigorosamente o padrão MVC, com separação clara entre camadas. A estrutura de diretórios é apresentada a seguir:

```
1 ifmg-crud-concessionaria/
2 |
3 |-- config/
4 |   |-- database.php
5 |
6 |-- src/
7 |   |-- Controllers/
8 |       |-- VeiculoController.php
9 |       |-- UsuarioController.php
10 |       |-- CategoriaController.php
11 |       |-- VendaController.php
12 |   |
13 |   |-- Models/
14 |       |-- Veiculo.php
15 |       |-- Usuario.php
16 |       |-- Categoria.php
17 |       |-- Venda.php
18 |   |
19 |   |-- Views/
20 |       |-- layouts/
21 |           |-- cabecalho.php
22 |           |-- rodape.php
23 |       |-- veiculos/
24 |           |-- listar.php
25 |           |-- criar.php
26 |           |-- editar.php
27 |       |-- categorias/
28 |           |-- index.php
29 |       |-- auth/
30 |           |-- login.php
31 |           |-- registro.php
32 |       |-- admin/
33 |           |-- dashboard.php
34 |
35 |-- public/
36 |   |-- css/
37 |       |-- style.css
38 |   |-- js/
```

```
39 |   |-- main.js
40 |   |-- img/
41 |   |-- uploads/
42 |   |-- index.php
```

Listing 5.1 – Árvore de diretórios do projeto

5.2 Descrição das Camadas

5.2.1 Camada de Configuração (config/)

Contém arquivos de configuração sensíveis, incluindo credenciais de acesso ao banco de dados, constantes do sistema e parâmetros de ambiente.

5.2.2 Camada de Controle (Controllers/)

Responsável por receber requisições HTTP, validar dados de entrada, invocar métodos dos Models e determinar qual View será renderizada. Implementa validações server-side e tratamento de erros. Inclui controladores dedicados para Veículos, Usuários e Categorias.

5.2.3 Camada de Modelo (Models/)

Encapsula toda a lógica de acesso ao banco de dados. Utiliza PDO com prepared statements para executar operações CRUD de forma segura e eficiente.

5.2.4 Camada de Visualização (Views/)

Contém templates HTML/PHP responsáveis pela apresentação dos dados ao usuário. Utiliza sintaxe alternativa do PHP para maior legibilidade.

5.2.5 Diretório Público (public/)

Único diretório acessível diretamente via navegador. O arquivo `index.php` atua como front controller, direcionando todas as requisições para os controllers apropriados.

6 Funcionalidades Implementadas

6.1 Sistema de Autenticação

6.1.1 Cadastro de Usuários

Implementa validações de CPF, e-mail único e força de senha. Utiliza hash bcrypt para armazenamento seguro de credenciais.

6.1.2 Login Seguro

Autenticação com verificação de senha através de `password_verify()`. Gerenciamento de sessões com tokens CSRF para prevenção de ataques de falsificação de requisição.

6.1.3 Controle de Acesso Baseado em Papéis

Diferenciação entre usuários administradores (acesso total) e usuários comuns (acesso restrito). Implementação de middleware de autorização em rotas protegidas.

6.2 Gestão de Veículos

6.2.1 Listagem com Filtros

Interface de listagem com recursos de:

- Busca por marca, modelo ou ano;
- Ordenação por preço, ano ou data de cadastro;
- Paginação de resultados;
- Filtros de faixa de preço e quilometragem.

6.2.2 Cadastro de Veículos

Formulário completo com validações client-side e server-side, incluindo:

- Upload múltiplo de imagens com validação de tipo e tamanho;
- Campos técnicos (marca, modelo, ano, cor, placa);
- Descrição de opcionais e características;
- Definição de preço e status (disponível/vendido).

6.2.3 Página de Detalhes

Visualização detalhada com carrossel de fotos, especificações técnicas e botão de contato via WhatsApp com mensagem pré-formatada.

6.2.4 Edição e Exclusão

Operações restritas a administradores, com confirmação de ações destrutivas via JavaScript e validação server-side.

6.2.5 Gestão de Categorias

Implementação de um módulo adicional para classificação dos veículos.

6.2.6 Cadastro e Listagem

Funcionalidade restrita ao painel administrativo que permite a criação rápida de novas categorias (ex: SUV, Hatch, Sedan) e visualização em lista.

6.2.7 Exclusão de Categorias

Permite que administradores removam categorias obsoletas do sistema, garantindo a organização dos dados.

6.3 Interface do Usuário

6.3.1 Design Responsivo

Layout adaptável implementado com Bootstrap 5, garantindo usabilidade em dispositivos móveis, tablets e desktops.

6.3.2 Tema Personalizado

Paleta de cores Dark/Yellow com CSS custom properties para facilitar manutenção e customização.

6.3.3 Interatividade Client-Side

JavaScript vanilla implementando:

- Máscaras de CPF e telefone;
- Validação em tempo real de formulários;

- Limpeza automática de espaços em branco (trim);
- Confirmações de ações críticas.

7 Configuração e Instalação

7.1 Requisitos do Sistema

7.1.1 Servidor Web

- PHP $\geq$ 8.0 com extensões PDO e pdo_pgsql;
- Apache 2.4+ ou Nginx 1.18+;
- PostgreSQL 12+ (obrigatório).

7.1.2 Ambiente de Desenvolvimento

- XAMPP, Laragon, WAMP ou servidor embutido do PHP;
- Navegador moderno (Chrome, Firefox, Edge);
- Editor de código (VS Code, PHPStorm).

7.2 Procedimento de Instalação

7.2.1 Etapa 1: Clonagem do Repositório

```
1 https://github.com/ghduque/ifmg-crud-concessionaria.git
2 cd ifmg-crud-concessionaria
```

Listing 7.1 – Comando para clonar o repositório

7.2.2 Etapa 2: Criação da Estrutura de Diretórios

```
1 # Criacao das pastas principais
2 mkdir -p config
3 mkdir -p src/{Controllers,Models,Views/{layouts,veiculos,categorias,auth,
   admin}}
4 mkdir -p public/{css,js,img,uploads}
5
6 # Criacao de arquivos base
7 touch config/database.php
8 touch src/Controllers/{VeiculoController.php,CategoriaController.php}
9 touch src/Models/{Veiculo.php,Categoria.php}
10 touch public/css/style.css
11 touch public/js/main.js
```

```
12 touch public/index.php
13
14 # Permissões de escrita (Linux/Mac)
15 chmod 777 public/uploads
```

Listing 7.2 – Comandos de criação da estrutura

7.2.3 Etapa 3: Configuração do Banco de Dados

Execute o script SQL apresentado no Capítulo 4 (Listagem 4.1) utilizando um cliente de banco de dados PostgreSQL (pgAdmin, DBeaver ou psql via terminal).

Observação: O sistema foi desenvolvido especificamente para PostgreSQL 12+ e utiliza tipos ENUM nativos do PostgreSQL.

7.2.4 Etapa 4: Configuração de Credenciais

Edite o arquivo `config/database.php` com suas credenciais:

```
1 <?php
2 $host = 'localhost';
3 $db   = 'db_autonivel';
4 $user = 'postgres';
5 $pass = 'sua_senha';
6 $port = '5432';
7
8 try {
9     $dsn = "pgsql:host=$host;port=$port;dbname=$db";
10    $pdo = new PDO($dsn, $user, $pass);
11    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
12    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
13 } catch (PDOException $e) {
14     die("Erro de conexao: " . $e->getMessage());
15 }
16 ?>
```

Listing 7.3 – Arquivo de configuração do banco de dados

7.2.5 Etapa 5: Inicialização do Servidor

Para ambientes de desenvolvimento, utilize o servidor embutido do PHP:

```
1 php -S localhost:8000 -t public
```

Listing 7.4 – Inicialização do servidor de desenvolvimento

Acesse o sistema através do navegador em: <http://localhost:8000>

8 Considerações Finais

O desenvolvimento do sistema AutoNível Multimarcas proporcionou a aplicação prática de conceitos fundamentais de engenharia de software, incluindo arquitetura MVC, segurança da informação e desenvolvimento full-stack. A opção por tecnologias nativas permitiu compreensão aprofundada dos mecanismos subjacentes ao desenvolvimento web moderno.

8.1 Contribuições do Projeto

- Implementação completa de arquitetura MVC sem frameworks;
- Aplicação de boas práticas de segurança (OWASP Top 10);
- Desenvolvimento de interface responsiva e acessível;
- Documentação técnica detalhada seguindo padrões ABNT.

8.2 Trabalhos Futuros

Como melhorias futuras, sugere-se:

- Implementação de API RESTful para integração com aplicativos móveis;
- Adição de sistema de notificações em tempo real (WebSockets);
- Implementação de relatórios gerenciais com gráficos interativos;
- Integração com gateways de pagamento;
- Implementação de testes automatizados (PHPUnit).

Referências

- [1] GAMMA, Erich et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1995.
- [2] PHP GROUP. **PHP Manual**. Disponível em: <https://www.php.net/manual/>. Acesso em: 15 jan. 2025.
- [3] OWASP FOUNDATION. **OWASP Top Ten 2023**. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 15 jan. 2025.
- [4] BOOTSTRAP TEAM. **Bootstrap Documentation**. Disponível em: <https://getbootstrap.com/docs/5.3/>. Acesso em: 15 jan. 2025.
- [5] POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 15 Documentation**. Disponível em: <https://www.postgresql.org/docs/15/>. Acesso em: 15 jan. 2025.
- [6] MOZILLA DEVELOPER NETWORK. **MDN Web Docs**. Disponível em: <https://developer.mozilla.org/>. Acesso em: 15 jan. 2025.